Audion Yandex Portable

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

Version 1.0.0 · 2026-08-23 · 81.4 MB

- Direct download — unmetered, no rate limits
- Project page — every version and how to install

`SHA-256: 80449aa2f3ecae1d20a94ad18531a7e164d6d0325b994833210353d506242894`

Builds a portable Yandex Browser, updates a build it is given, and keeps Chrome++ current — the piece the portability itself rests on. Nothing is installed into Windows: the full installer is unpacked rather than run.

Why: Yandex Browser trusts the Russian Ministry of Digital Development CA out of the box, so sites issued against it open without touching the system certificate store. A portable build adds the other half — the browser is separated from the system and travels as a folder.

How it works

The full Yandex installer is a PE whose resource section holds exactly one file, and that file holds the browser:

```
Yandex.exe (full_installer, ~200 MB)
└── browser.7z
    └── Browser-bin\        →  <build>\App\
          browser.exe
          <version>\browser.dll and the rest
```

Chrome++ provides the portability: its `version.dll` goes beside `browser.exe`. `browser.exe` imports `VERSION.dll`, that name is not in `KnownDLLs`, so the hijack takes and the running process carries `--portable` and `--user-data-dir`. The profile lands in `Data` beside `App`, the cache in `Cache`.

The wrapper is a choice: Chrome++, the proxy library (neyrostalker/proksi-biblioteka on GitFlic, pulled off the public pages without a token). The proxy library blocks registry writes instead of wiping the branch on exit and draws no complaint from Microsoft; it ships x86 and x64 only. The three engines and the VirusTotal check are covered in `docs/CHROME_PLUS_AND_DEFENDER.md`.

Chrome++ is a long-standing, respected open-source project; antivirus sometimes mistakes its `version.dll` for a threat. Why that is a false positive and how the program works around it during a build — see CHROME_PLUS_AND_DEFENDER.md.

A finished build:

```
Yandex Browser Portable\
  App\                          browser, version.dll, chrome++.ini
  Data\                         profile
  Cache\                        cache
  Yandex Browser Portable.cmd   launcher
  Portable-Build.json           which versions are inside
```

The interface

The root window is a switcher of three tabs: `Install`, `Update`, `Service`. A command with parameters unfolds on the tab itself: its own run button, named after the action, and its own fields. Service operations sit in a strip above the tabs. Choices are buttons — the chosen one washed with translucent blue, the rest outlined. Captions are short; the explanation lives in the tooltip.

Commands

Tab	Command	What it does
`Install`	`Build`	Downloads the installer and Chrome++, publishes a build into the Target folder.
`Update`	`Check`	Compares the published versions with the build. Downloads nothing.
`Update`	`Update`	Replaces `App` in the build Source points at, keeps `Data` and `Cache`.
`Update`	`Chrome++`	Replaces `version.dll` and `chrome++.ini`, leaves the browser alone.
`Service`	`7-Zip`	Checks the unpacker and puts a portable copy into the project folder.

Updating

The published version is read **before** downloading: `browser.yandex.ru` answers with a redirect to the CDN and the version sits in the path (`.../browser/yandex/26_6_5_621_113843/ru/Yandex.exe`). So a check costs one request, and the 200 MB are fetched only when the versions differ.

The build's own version is read out of the folder — `FileVersion` of `browser.exe` and of `version.dll` — so a build assembled elsewhere can be updated too. Version comparison knows that a release tagged `1.18.2` ships a file calling itself `1.18.2.0`.

The update happens **in place**: the build is refreshed in the folder Source points at rather than published anew into the Target folder, which is there for new builds. With nothing in Source, the program looks in `output\Portable`. `App` is swapped by renaming — the old folder steps aside, the new one takes its place, and only then is the old one deleted. When the browser is running and the rename fails, the operation says so and leaves the build alone.

Chrome++ is refreshed together with the browser and by its own command. The asset list comes from the GitHub API and, on any error from it, off the `releases/latest` and `releases/expanded_assets/<tag>` pages, where there is no quota: 60 anonymous API calls an hour run out quietly.

What it leaves in the system

A portable build is not installed, but the browser keeps counters in `HKCU\Software\Yandex\YandexBrowser`, and those never reach the profile. So the build wipes that

branch on exit by default: the command is written into Chrome++'s `launch_on_exit` and runs when the browser really terminates.

The branch is shared with a normally installed Yandex Browser. If one is present on the same machine, clear the `Leave no traces in Windows` checkbox.

The bundled `service_update.exe` is removed from the build: it would otherwise pull an installation into the system past the portable folder.

Requirements

- Windows, the portable Python in `runtime\` (ships with the project).
- `tools\7zip\bin\7za.exe` — installed by the `7-Zip` command on the `Service` tab.
- About 200 MB of download and up to 1 GB while unpacking and publishing.

Running

```
launcher_gui.cmd
```